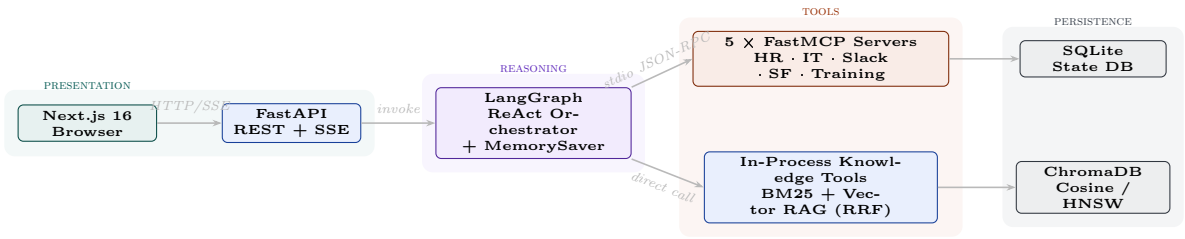
Employee Onboarding AI Agent

Solution Design Document Acme Corp · April 2026



Architecture

The diagram above shows the four layers. The browser communicates with a FastAPI backend that streams responses as Server-Sent Events (SSE). A LangGraph `create_react_agent` with `MemorySaver` is the reasoning core. It dispatches work to five FastMCP subprocess servers (simulating enterprise systems) via stdin/stdout JSON-RPC, and to in-process LangChain knowledge tools backed by ChromaDB. All structured state (employees, tickets, approvals) lives in SQLite.

AI Agent Application

The agent operates in a *Reason* → *Act* → *Observe* loop, selecting tools based on the new hire's `role`, `level`, and `department`:

- **Profile enrichment** — updates the HR record; provisions Salesforce and Slack accounts with role-specific fields.
- **Action execution** — opens IT tickets for hardware/software; assigns mandatory training modules matched to seniority.
- **Policy Q&A** — answers freeform questions by retrieving semantically relevant chunks from the knowledge base with source attribution.
- **Approval workflows** — creates and tracks approval requests in-conversation.
- **Error handling** — each tool wraps calls in try/except and returns structured error strings; the agent re-plans without crashing.

RAG Pipeline (3 production techniques)

1. **Cosine similarity** — ChromaDB collection configured with `hnsw:space=cosine` so relevance is magnitude-independent.
2. **Contextual chunking** — each chunk is prefixed with its document title and section header before embedding, preserving meaning in isolation.
3. **Hybrid retrieval** — BM25 keyword search and vector semantic search merged via Reciprocal Rank Fusion (equal 0.5/0.5 weighting with `EnsembleRetriever`).

Orchestration

Request flow: User sends `POST /api/chat/{id}` → FastAPI hands off to the LangGraph orchestrator → agent issues tool calls (MCP servers or RAG) → results are observed and used for the next reasoning step → final text is streamed back as SSE and rendered as markdown in the browser.

MCP communication: Each of the five servers is a subprocess spawned at startup. `langchain-mcp-adapters` serialises tool calls to JSON-RPC over stdin/stdout, making them indistinguishable from native LangChain tools to the agent.

Memory: `MemorySaver` keeps per-employee conversation history keyed by `thread_id`, enabling multi-turn context without re-querying the database each turn.

Trade-offs & Considerations

Assumptions & Trade-offs

- **In-process RAG** — ChromaDB's embedded SQLite is unsafe for cross-process access on Windows. Knowledge tools run in the main process, sacrificing isolation to avoid file-lock deadlocks.
- **SQLite** — appropriate for development; production requires PostgreSQL with connection pooling.
- **Mock MCP servers** — simulate Workday, ServiceNow, Slack, Salesforce, and an LMS. Production integration needs OAuth, rate limiting, and idempotency guards.
- **In-RAM checkpointer** — `MemorySaver` is single-process only; multi-instance deployments need `PostgresSaver` or a Redis-backed checkpointer.

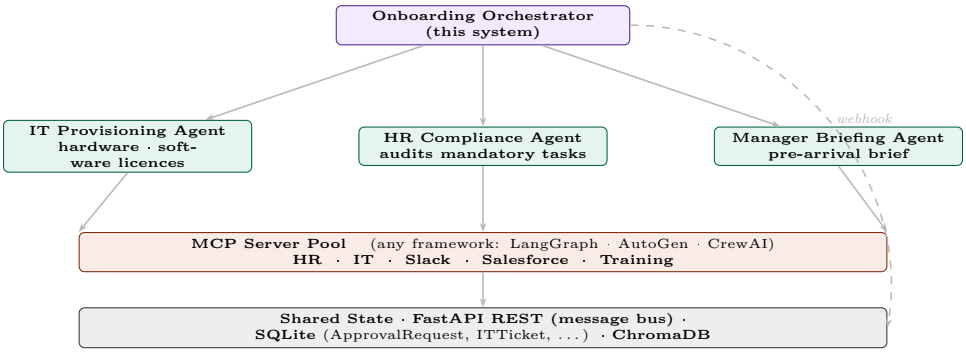
Scalability

The FastAPI layer is stateless per request. Horizontal scaling requires an external checkpointer. The BM25 index rebuilds at startup — fine for a static knowledge corpus; a dynamic knowledge base needs incremental indexing. ChromaDB should be replaced by a managed vector store (Pinecone, Weaviate, or PostgreSQL `pgvector`) at production scale.

Security & Governance

- All responses are scoped to the requesting `employee_id`.
- Sensitive fields (salary, SSN) should be excluded via field-level access control before reaching LLM context.
- All tool invocations are audit-logged via `structlog`; these should be shipped to a SIEM.
- The `POST /api/admin/reset-db` endpoint must be gated by RBAC before deployment.
- The Ollama path (self-hosted models) satisfies data-residency and air-gapped deployment requirements.

Multi-Agent Collaboration via MCP



Because each MCP server exposes a standard JSON-RPC interface, any agent framework can attach to the same tool pool without modification. Agents coordinate through shared SQLite state and FastAPI REST endpoints, which act as a neutral message bus. The orchestrator can delegate sub-tasks to specialist agents and receive completion signals via webhook callbacks.

Stack: Python 3.12 · FastAPI · LangGraph · LangChain · ChromaDB · FastMCP · Next.js 16 · SQLite · Claude Sonnet 4.6 / Ollama